



CS 498: Transformers Deep Dive

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

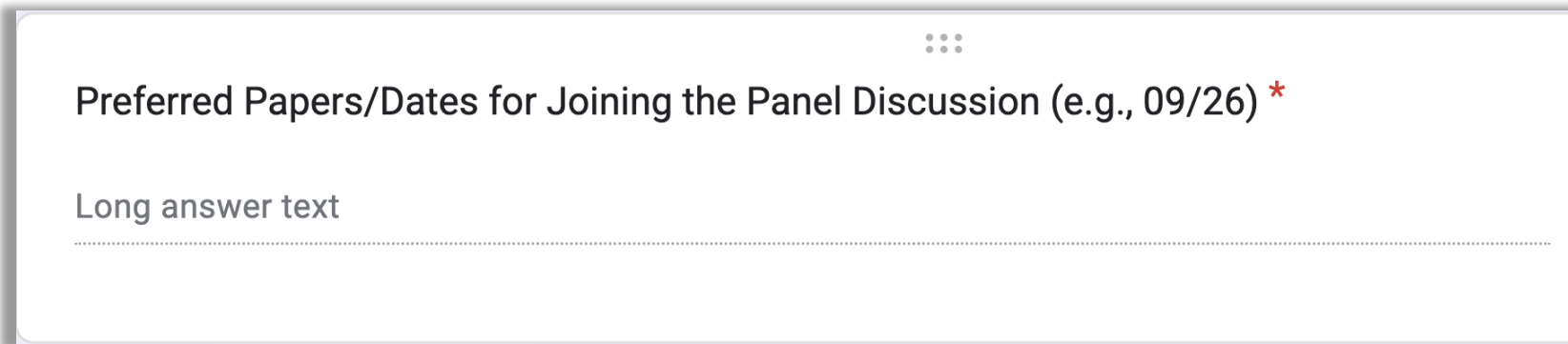
- **Team registration**
 - 4-5 members in a team by Sept 8



Preferred Papers/Dates for Joining the Panel Discussion (e.g., 09/26) *

Long answer text

- **Team registration**
 - 4-5 members in a team by Sept 8



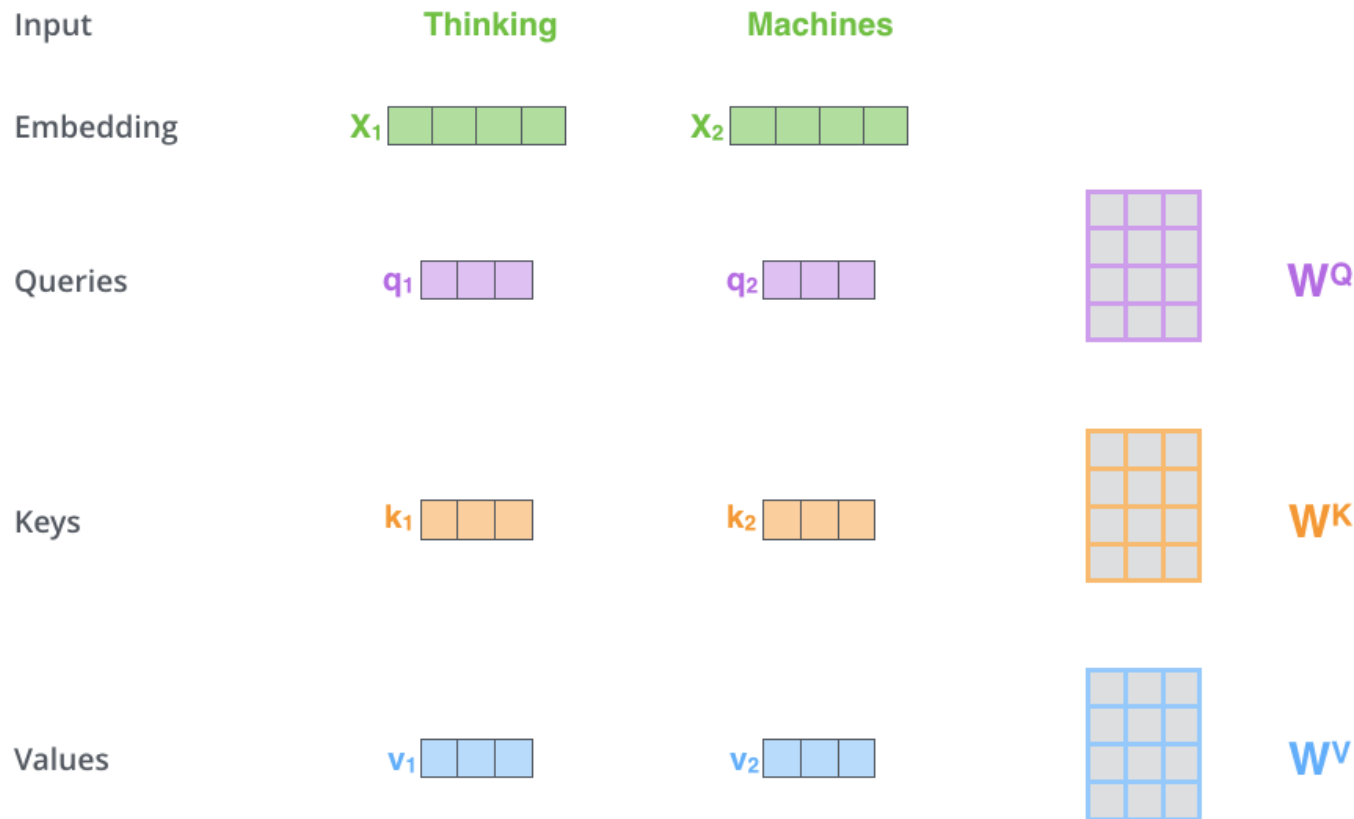
A screenshot of a registration form with a light gray border and rounded corners. At the top center, there are three small gray dots. Below them, the text "Preferred Papers/Dates for Joining the Panel Discussion (e.g., 09/26) *" is displayed in a dark gray font. Underneath this, the text "Long answer text" is shown in a lighter gray font, followed by a horizontal dotted line indicating a text input area.

- **Course registration (3 or 4 credits)?**
 - One more assignment for 4-credit students
- **Up to two absences since your registration**

Recall: Self-Attention in Transformers



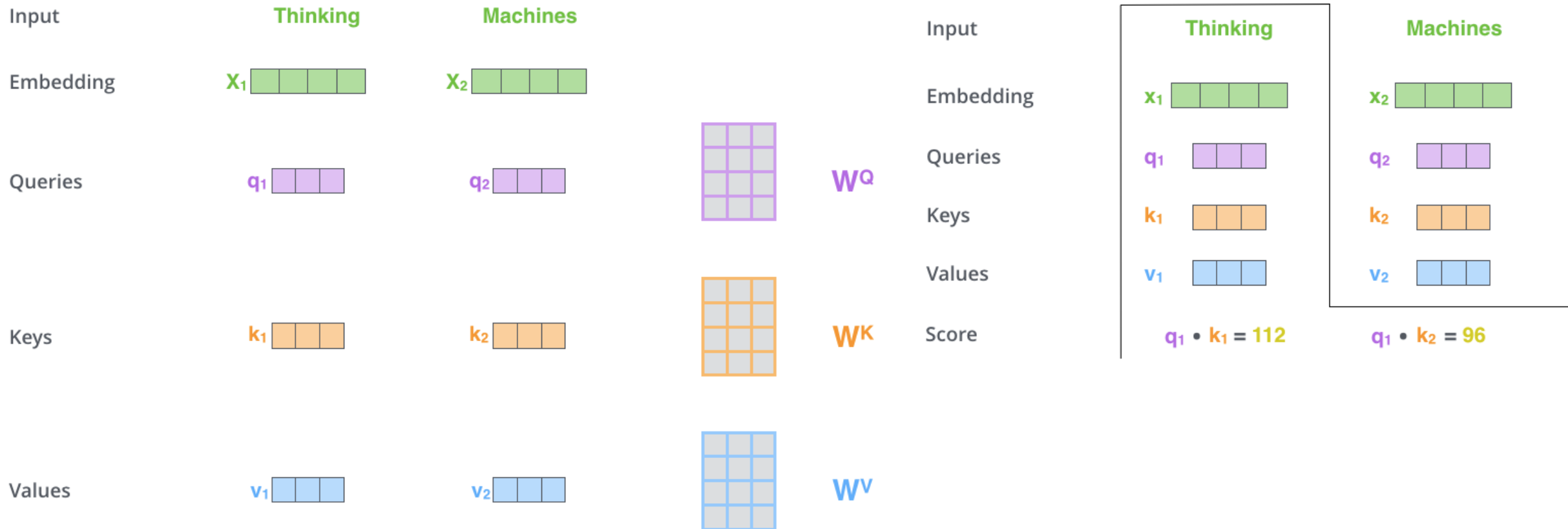
- **Step 1:** compute “key, value, query” embedding for each input token



Recall: Self-Attention in Transformers



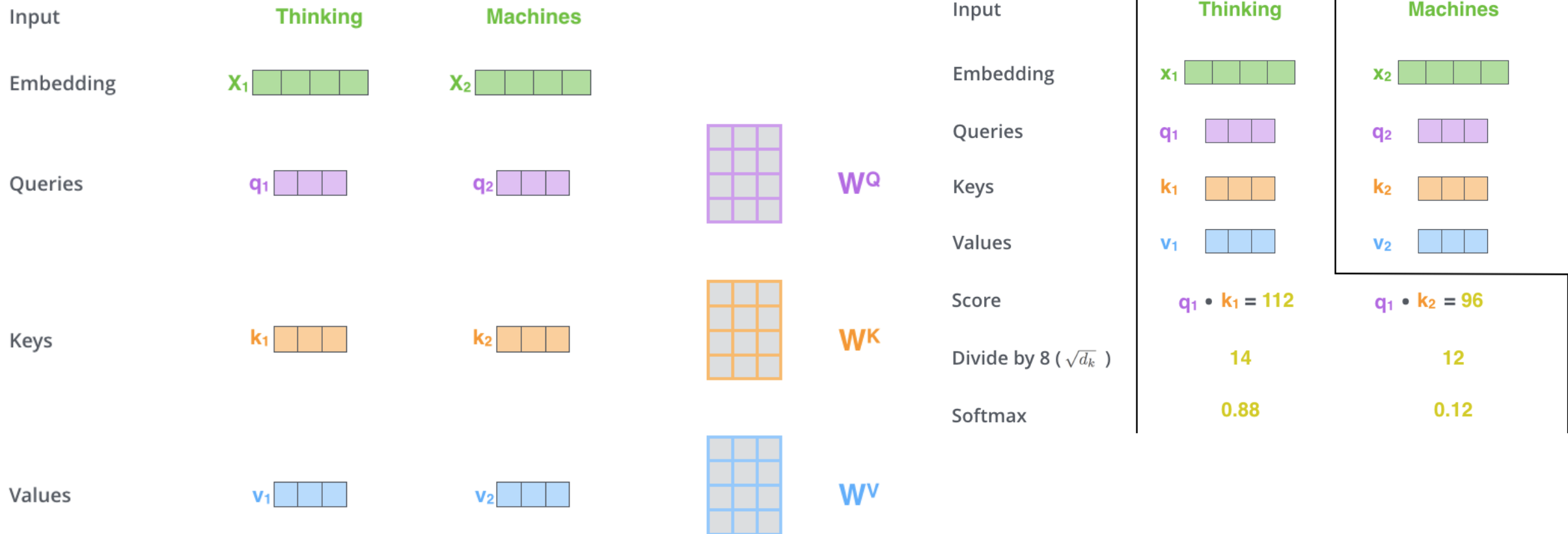
- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens



Recall: Self-Attention in Transformers



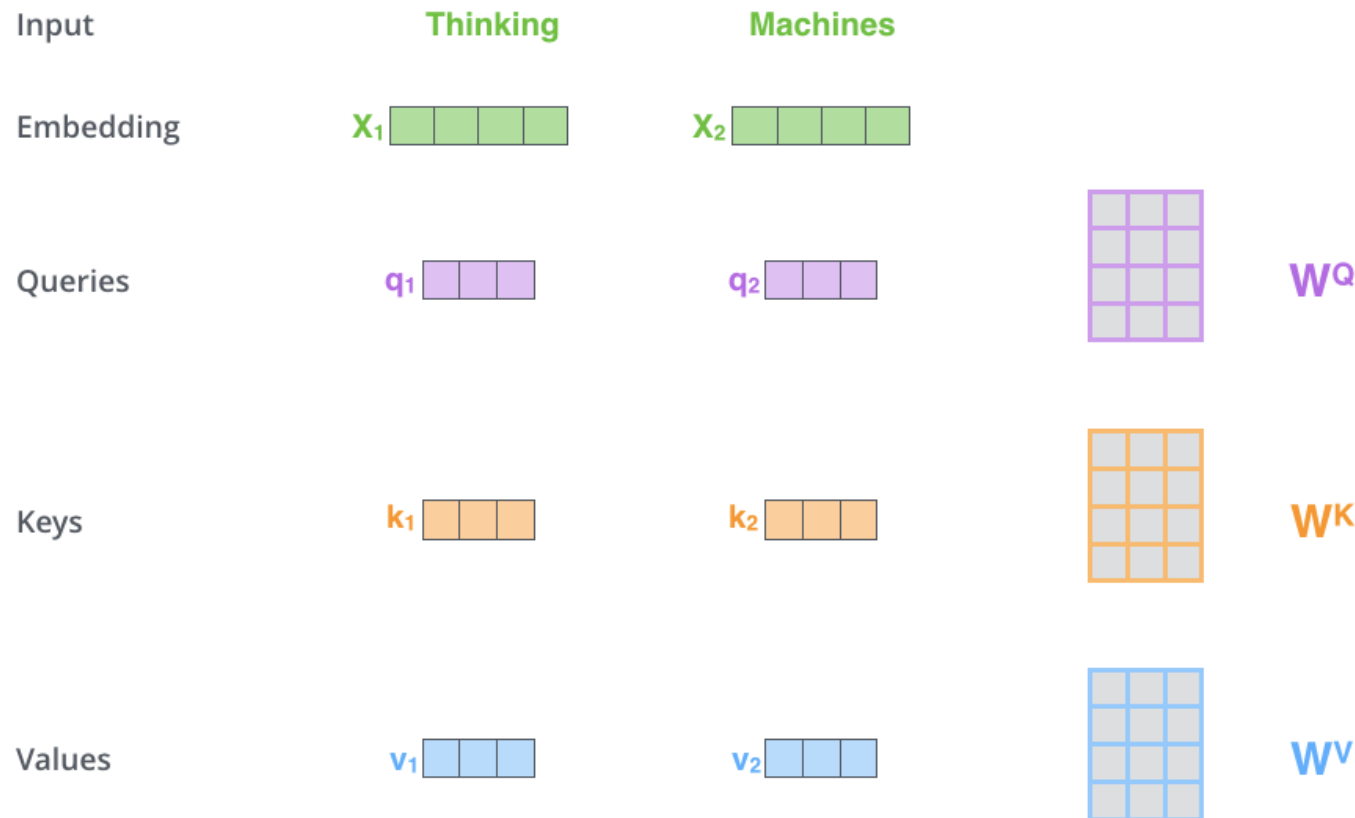
- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens
- **Step 3:** compute normalized attention scores



Recall: Self-Attention in Transformers



- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens
- **Step 3:** compute normalized attention scores
- **Step 4:** get new representation by weighted sum of values



Input

Embedding

Queries

Keys

Values

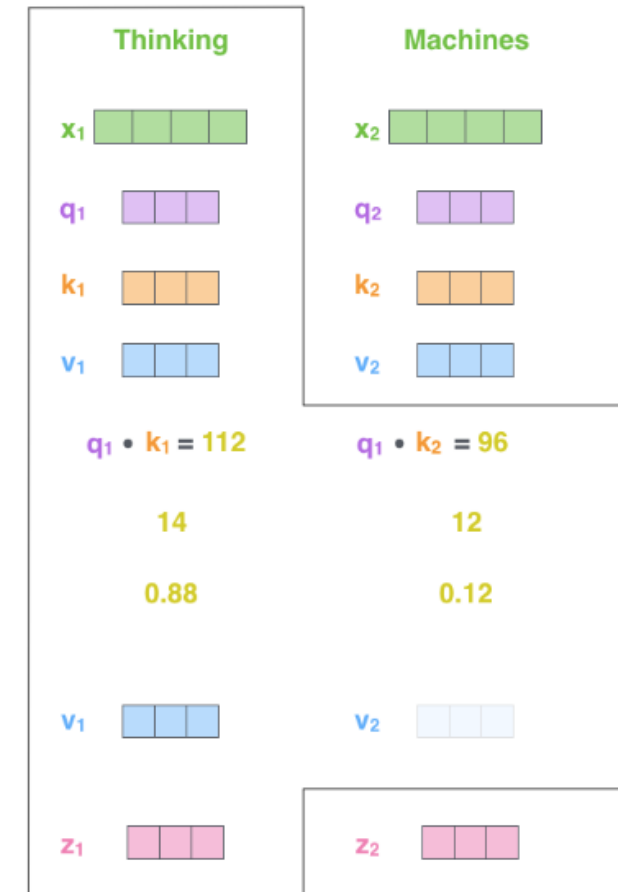
Score

Divide by 8 ($\sqrt{d_k}$)

Softmax

Softmax
X
Value

Sum

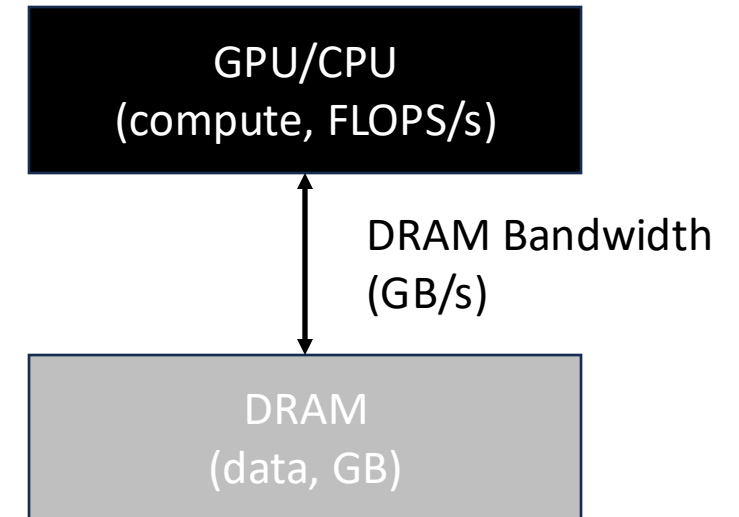


- **Transformers Deep Dive**

- Performance Analysis (e.g., Arithmetic Intensity)
- Multi-head & Multi-query Attention
- Native Sparse Attention (ACL'25 Best Paper Award)

Arithmetic Intensity (AI): evaluate the efficiency of computational algorithms

- $AI = \#ops / \#bytes$

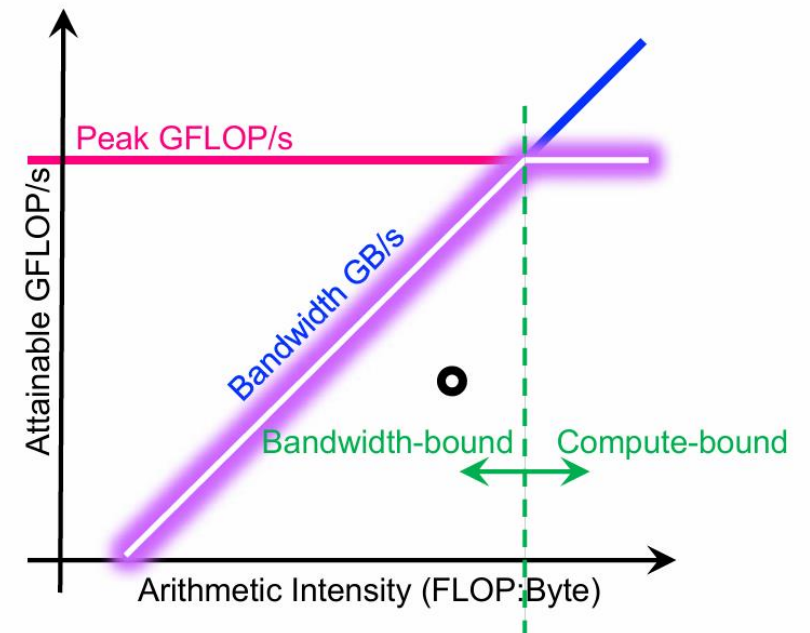


Arithmetic Intensity (AI): evaluate the efficiency of computational algorithms

- $AI = \#ops / \#bytes$

System performance is bound by

- The peak compute FLOPS (Floating-Point Ops)
 - The memory bandwidth
- Use intensity analysis to guide system optimization and hardware development



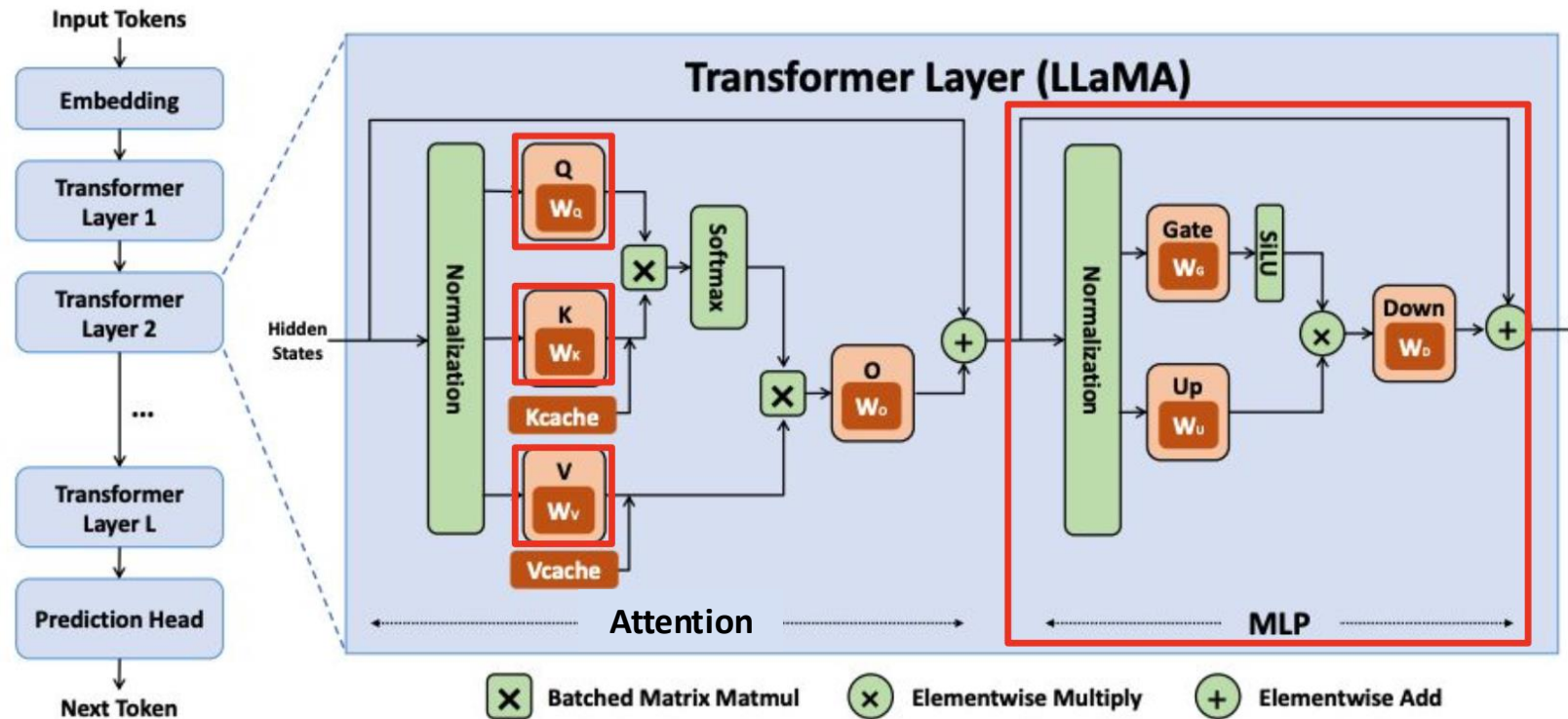
```
void add(int n, float* A, float* B, float* C){  
    for (int i=0; i<n; i++)  
        C[i] = A[i] + B[i];  
}
```

Two loads, one store per math op

1. Read A[i]
2. Read B[i]
3. Add A[i] + B[i]
4. Store C[i]

Arithmetic intensity = $1/3$

Arithmetic Intensity: Transformers



Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- Step 1: $\mathbf{Q}$ matrix $(N * h)$ x $\mathbf{K}^T$ matrix $(h * N)$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

A				B			
1	2	3	4	1	2	3	4
5	6	7	8	5	6	7	8
9	10	11	12	9	10	11	12
13	14	15	16	13	14	15	16

$c_{1,1}$	$c_{1,2}$		
$c_{2,1}$	$c_{2,2}$		

$$C_{i,j} = A_{(i, \cdot)} \times B_{(\cdot, j)} : \underbrace{1 \times 1 + 2 \times 5 + 3 \times 9 + 4 \times 13}_{\text{h-element mul \& add} \rightarrow 2 * h \text{ Flops}}$$

$$C = A \times B$$

Compute Flops:

- Step 1: Q matrix $(N * h)$ x K^T matrix $(h * N)$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- Step 1: $\underbrace{Q \text{ matrix}_{(N \times h)} \times K^T \text{ matrix}_{(h \times N)}}_{\text{Flops: } 2 * h * N * N}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Loads/stores: $\underbrace{(2 \cdot d \cdot N + N \cdot N)}_{\text{Step 1}} \cdot 2$ (due to mul/add ops)

Compute Flops:

- $\underbrace{2 \cdot h \cdot N \cdot N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 \cdot N \cdot N}_{\text{Step 2 (softmax)}} + \underbrace{2 \cdot N \cdot N \cdot h}_{\text{Step 3}}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Loads/stores: $\underbrace{2 * 2 * d * N + 2 * N * N + 2 * N * N}_{\text{Step 1}} + 2 * (N * N + N * d) + 2 * N * d$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

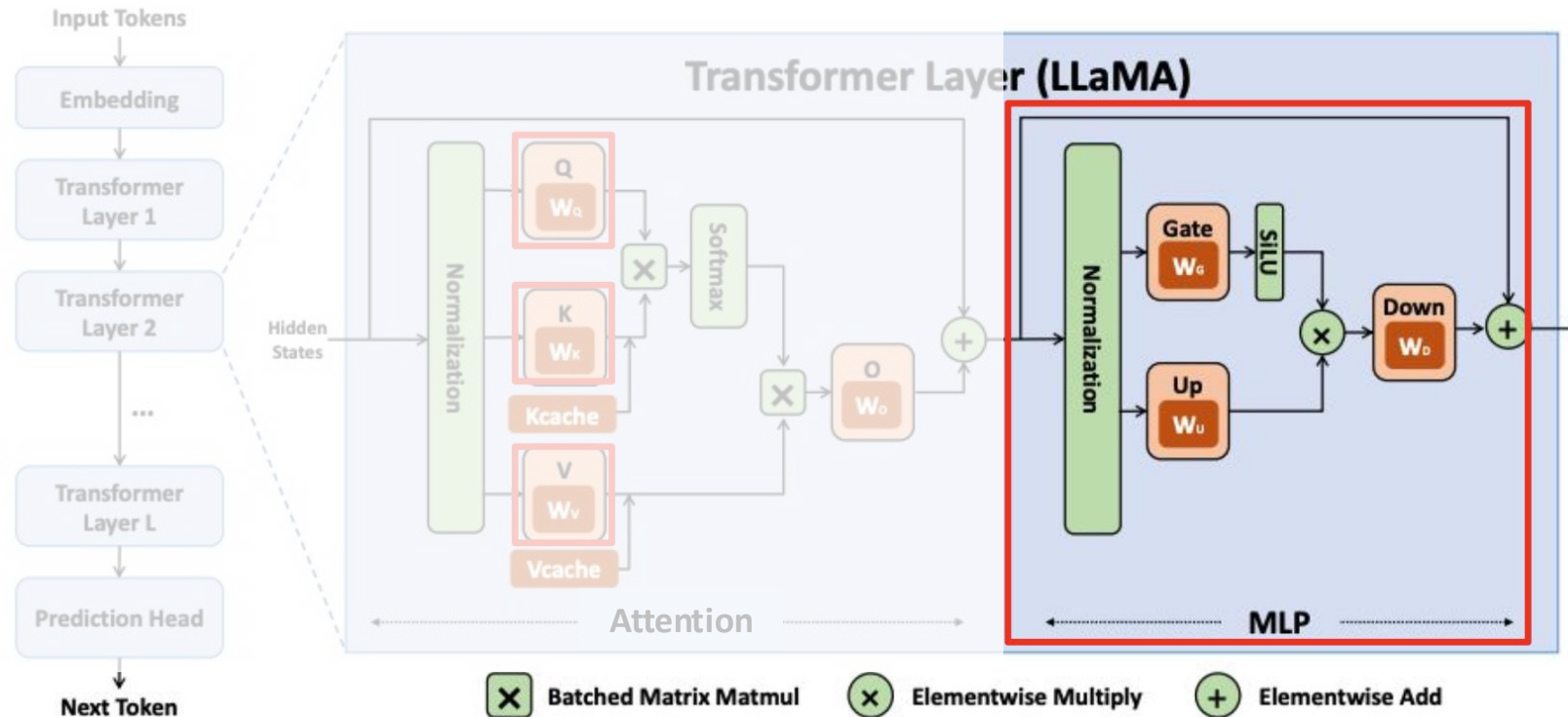
Compute Flops:

$$\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$$

Attention Compute: $(4 * h + 3) * N^2$; Memory IO: $8 * N^2 + 8 * N * d$

Loads/stores: $\underbrace{2 * 2 * d * N + 2 * N * N + 2 * N * N + 2 * N * N}_{\text{Step 1}} + 2 * (N * N + N * d) + 2 * N * d$

Arithmetic Intensity: Transformers (MLP)



Linear layer, essentially a GEMM: X
 $\cdot W$

- Shape: $X(N, K)$, $W(K, M)$

Compute: $2 \cdot N \cdot K \cdot M$

Memory: $2 \cdot N \cdot K + 2 \cdot K \cdot M + 2 \cdot N \cdot M$

MLP FLOPs/Memory ops = 1365 ops/byte

- $N = K = M = 4096$

A10 GPU Analysis:

- Compute capability: 125TF, Memory bandwidth: 600GB/s
- Ops/byte = $125\text{TF} / 600\text{GB/s} = 208.3 \text{ ops/byte}$

MLP FLOPs/Memory ops = 1365 ops/byte

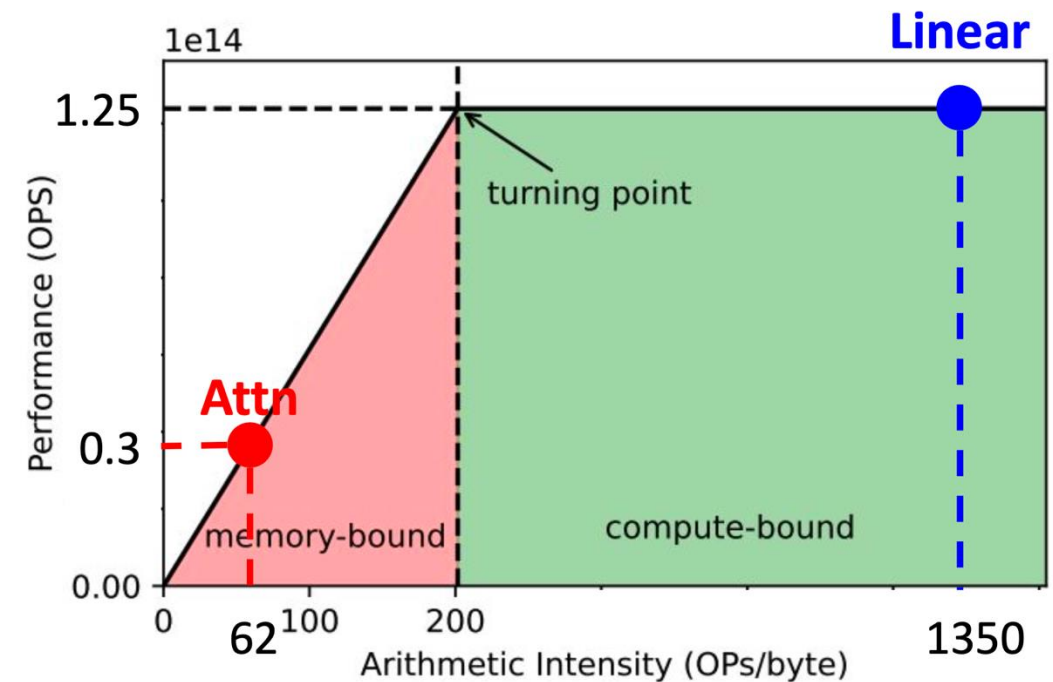
- $N = K = M = 4096$

A10 GPU Analysis:

- Compute capability: 125TF, Memory bandwidth 600GB/s
- Ops/byte = $125\text{TF} / 600\text{GB/s} = 208.3 \text{ ops/byte}$

MLP's AI is much higher than ~200 ops/byte

- Compute bound



Short input length (N)

Attention Compute: $(4h + 3) * N^2$; Memory IO: $8 * N^2 + 8 * N * d$

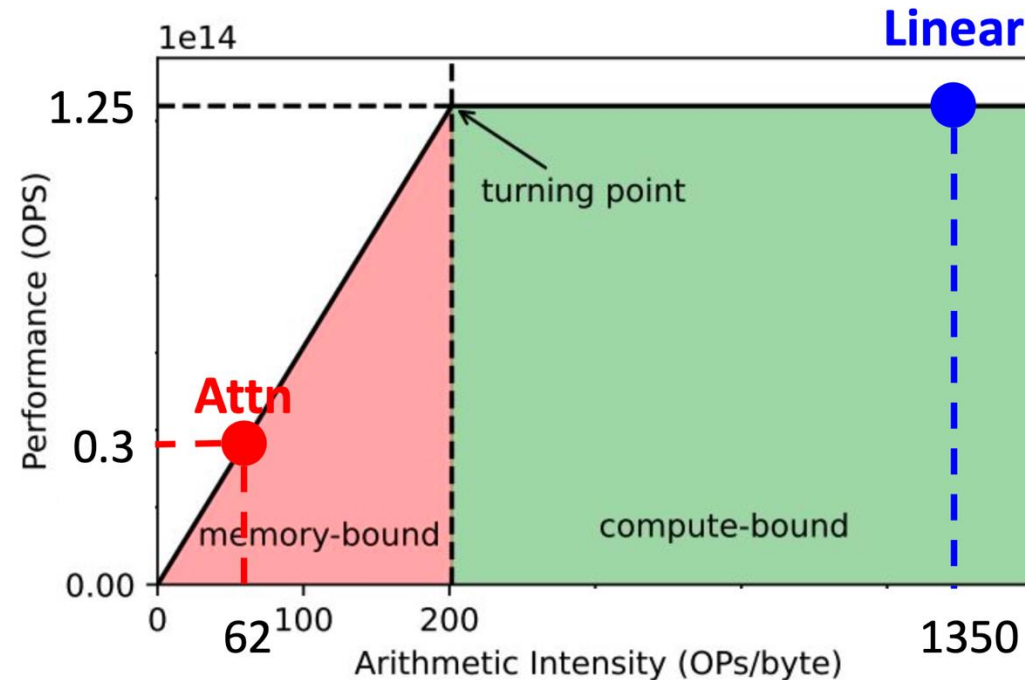


$$\text{Arithmetic Intensity} = \{(4h + 3) * N^2\} / \{8 * N^2 + 8 * N * d\}$$



Arithmetic intensity increases as the input length N grows.

Attention Compute: $(4h + 3) * N^2$; Memory IO: $8 * N^2 + 8 * N * d$



Attention can become compute-bound as N grows

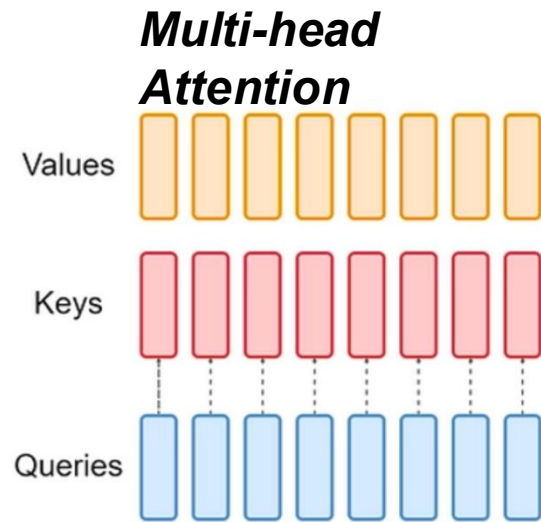
Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot d$

How to speed up?

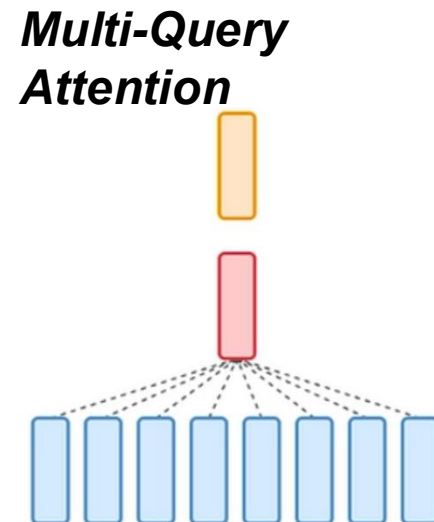
- Do many attention head calculation *in parallel, and combine*
- Each head has its *own* set of parameters
- Different heads can learn different “interactions” between inputs



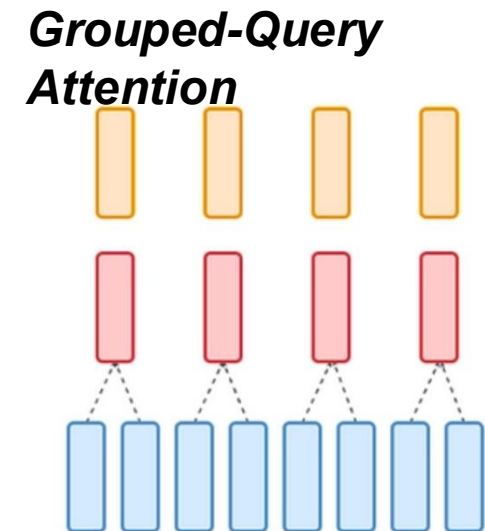
Parallel execution is good, but *memory footprint* is still large



Each head digests QKV separately



Share single key and value heads across query heads



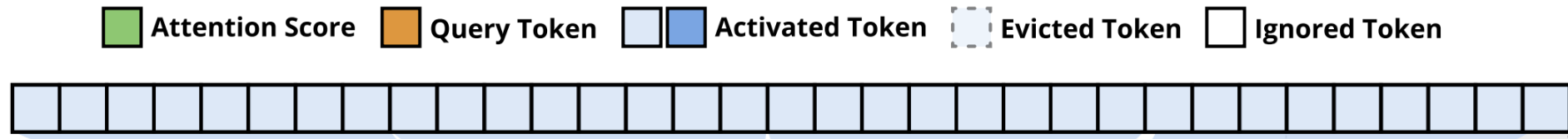
*Share single key and value heads for **each group** of query heads*

Better compute, smaller memory footprint, but quality may drop

Native Sparse Attention (ACL'25 from DeepSeek)



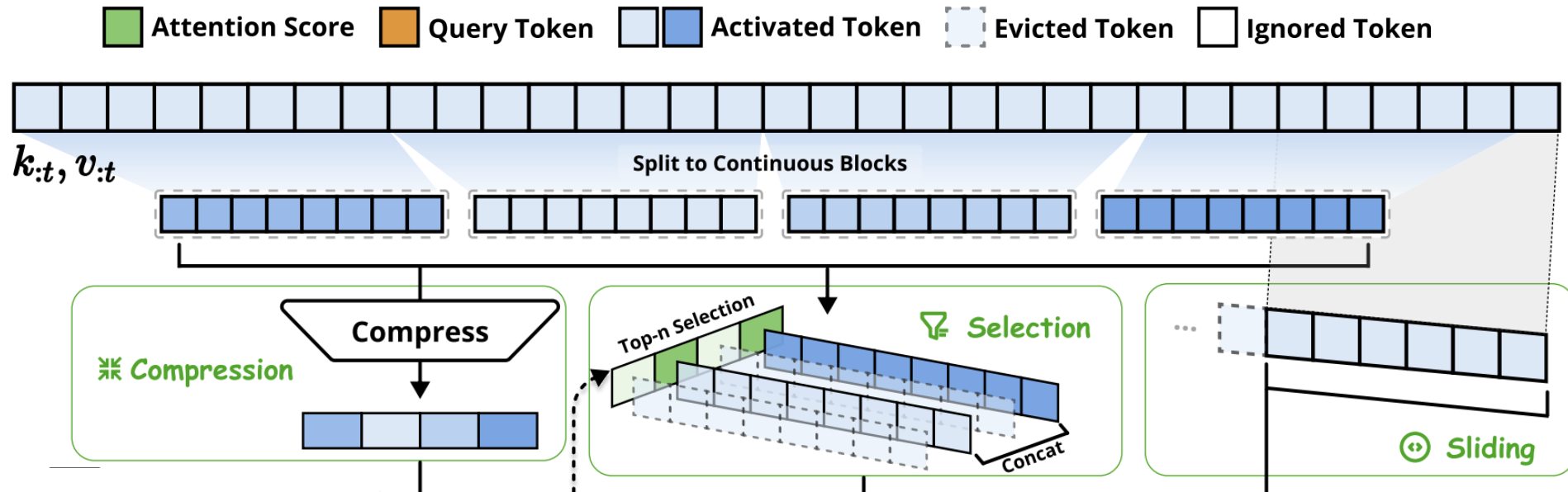
- Key insight: we can summarize long-context input, identify and focus on key words



Native Sparse Attention (ACL'25 from DeepSeek)



- Key insight: we can summarize long-context input, identify and focus on key words

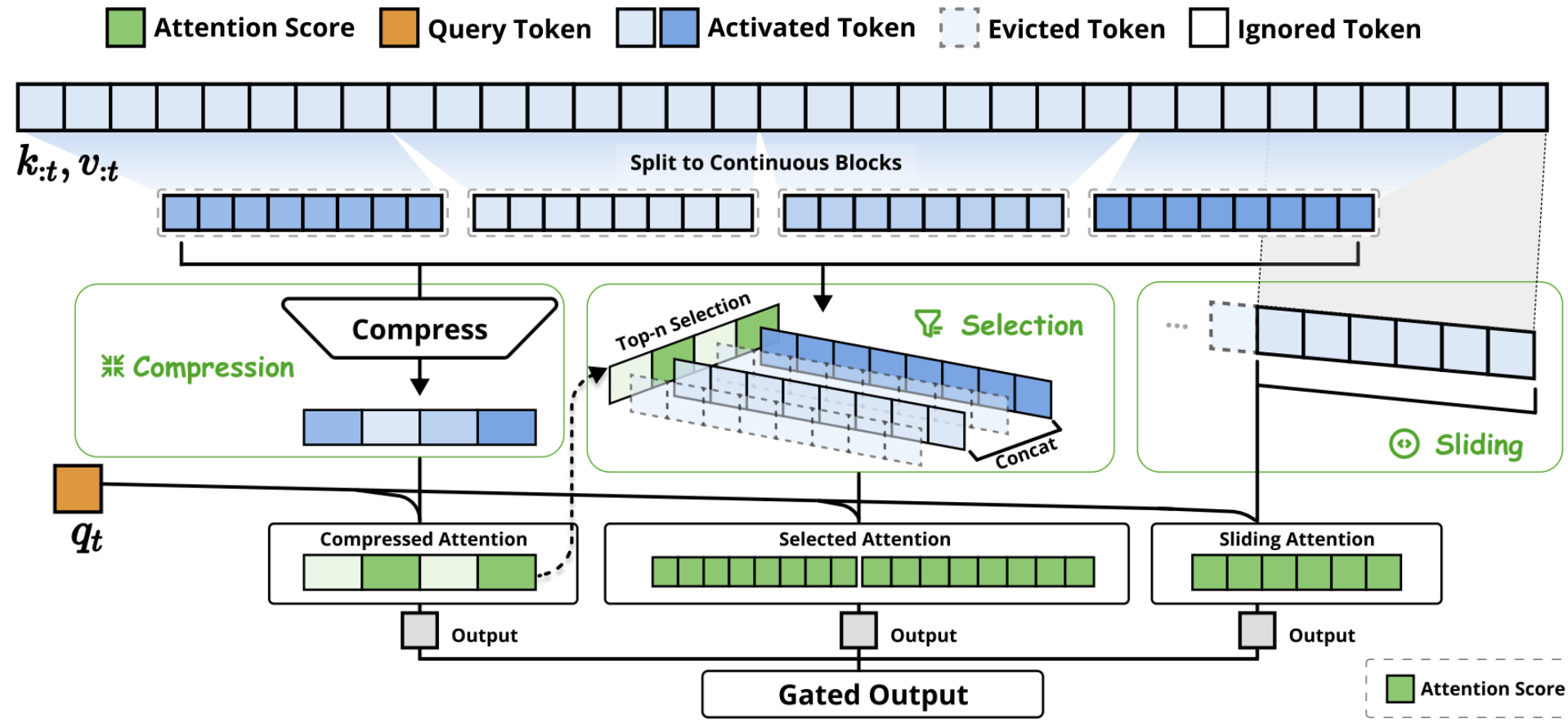


[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

Native Sparse Attention (ACL'25 from DeepSeek)



- Key insight: we can summarize long-context input, identify and focus on key words



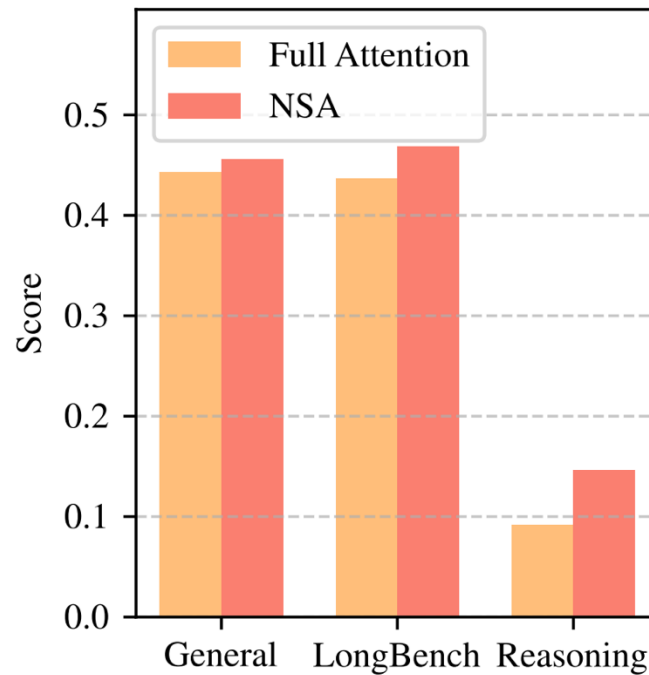
[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

Native Sparse Attention Performance

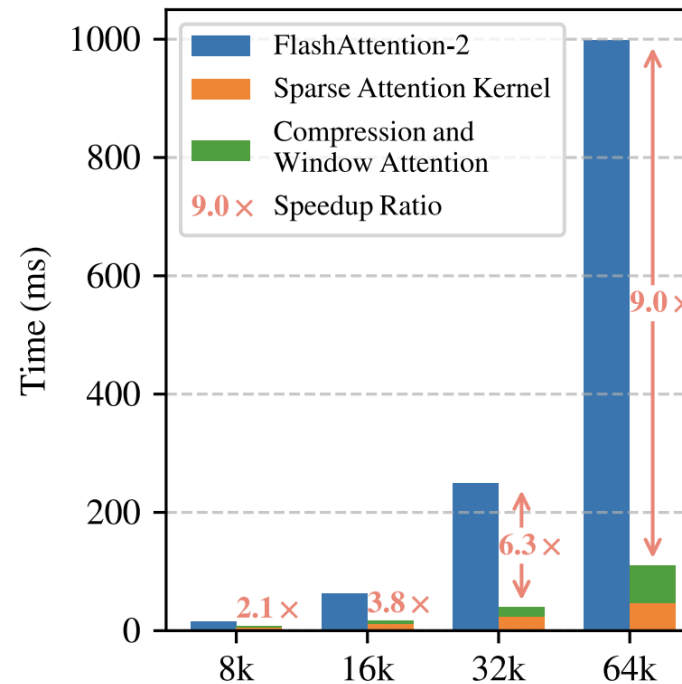


- Better quality, compute, and memory footprint

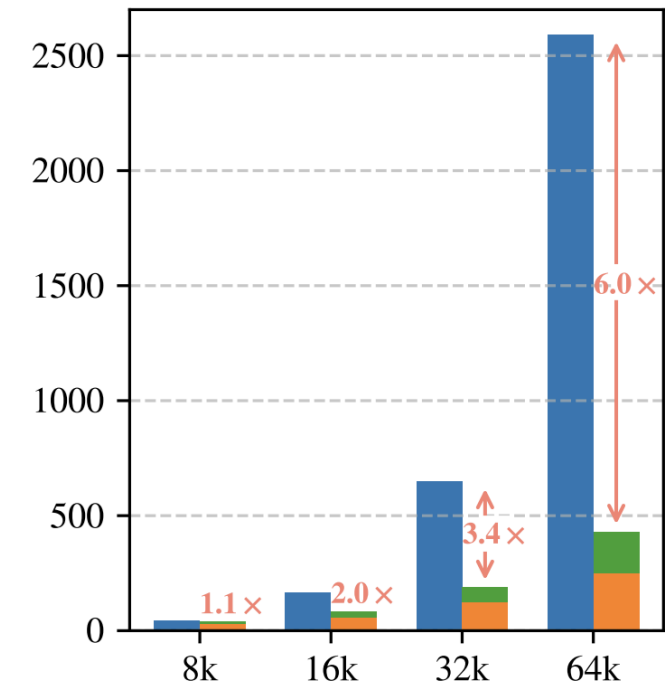
Performance on Benchmarks



Forward Time Comparison



Backward Time Comparison



Input length

[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

- **Transformers Deep Dive**

- Performance Analysis (e.g., arithmetic intensity)
 - Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot d$
- Multi-head & Multi-query Attention
 - Parallel execution of heads
- Native Sparse Attention (ACL'25 Best Paper Award)
 - Summarize, focus on key tokens and neighboring tokens

- **Transformers Deep Dive**

- Performance Analysis (e.g., arithmetic Intensity)
 - Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot d$
- Multi-head & Multi-query Attention
 - Parallel execution of heads
- Native Sparse Attention (ACL'25 Best Paper Award)
 - Summarize, focus on key tokens and neighboring tokens

- **Paper summary by Sept 4**



DeepSeek-V3 Technical Report

DeepSeek-AI

research@deepseek.com

- **Each summary (4-5 paragraphs) must follow the template**
 - What is the problem and why is it important?
 - The main proposed idea(s).
 - A summary of your understanding of different components of the proposed technique, e.g., the purpose of critical design choices.
 - What are the technical drawbacks/limitations
 - How will you improve them?

Grading criteria, each summary has 10 points in total. For each item here, you get:

- 2: The summary item demonstrates a clear understanding of the paper.
- 1: The summary item misses the point of the paper.
- 0: The summary item is missing.